ЛАБОРАТОРНА РОБОТА № 1

Тема: Робота з базовими типами даних

Мета: Ознайомитись з основними можливостями мови програмування Go, найпростішої структурою програми, з властивостями базових типів. Отримати практичні навички по роботі з базовими типами даних в мові Go.

хід роботи

Завдання 1.

1. Вивести текст українською мовою

Виконання завдання:

```
func main() {
   // Оголошення змінної
   var str string = "Golang!"

   // Виведення в консоль тексту
   fmt.Println("Hello ", str)

   //Завдання.
   // 1. Вивести текст українською мовою
   fmt.Println("Привіт ", str)
}
```

Результат виконання:

```
Hello  Golang!
Привіт  Golang!

Process finished with the exit code 0
```

Рис.1.1

Завдання 2.

1. Створити цілочисельну змінну (результат не відображати)

Виконання завдання:

```
func main() {
    // Усі змінні ініціалізуються значенням за замовчуванням, тобто 0
    var defaultInt8 int8
    var defaultInt16 int16
    var defaultInt32 int32
    var defaultInt64 int64
    var defaultInt int

    fmt.Println("Default values (signed): ", defaultInt8, defaultInt16, defaultInt32,
defaultInt64, defaultInt)

    var defaultuInt8 uint8
    var defaultuInt16 uint16
    var defaultuInt32 uint32
    var defaultuInt64 uint64
    var defaultuInt uint

    fmt.Println("Default values (unsigned): ", defaultuInt8, defaultuInt16, defaultuInt32,
defaultuInt64, defaultuInt)

    // Завдання.
    //1. Створити цілочисельну змінну (результат не відображати)
    var myInt int = 42
```

```
    _ = myInt // Використовуємо змінну, але не виводимо результат
}
```

Результат виконання:

```
Default values (signed):  0 0 0 0 0
Default values (unsigned):  0 0 0 0 0

Process finished with the exit code 0
```

Рис. 1.2

Завдання 3.

1. Вивести типи всіх змінних

2. Присвоїти змінній intVar змінні userinit16 і userautoinit. Результат вивести в консоль.

Виконання завдання:

```
func main() {
    // Ініціалізація змінних
    var userinit8 uint8 = 1
    var userinit16 uint16 = 2
    var userinit64 int64 = -3
    var userautoinit = -4 // Такий варіант ініціалізації також можливий

    fmt.Println("Values: ", userinit8, userinit16, userinit64, userautoinit, "\n")

    // Короткий запис оголошення змінної
    // тільки для нових змінних
    intVar := 10

    fmt.Printf("Value = %d Type = %T\n", intVar, intVar)

    // Завдання.
    // 1. Вивести типи всіх змінних
    fmt.Printf("userinit8 Type = %T\n", userinit8)
```

```
fmt.Printf("userinit16 Type = %T\n", userinit16)
fmt.Printf("userinit64 Type = %T\n", userinit64)
fmt.Printf("userautoinit Type = %T\n", userautoinit)
fmt.Printf("intVar Type = %T\n\n", intVar)

// 2. Присвоїти змінній intVar змінні userinit16 і userautoinit. Результат вивести в консоль.
intVar = int(userinit16)
fmt.Printf("intVar = userinit16: Value = %d Type = %T\n", intVar, intVar)

intVar = int(userautoinit)
fmt.Printf("intVar = userautoinit: Value = %d Type = %T\n", intVar, intVar)
}
```

Результат виконання:

```
Values:  1 2 -3 -4

Value = 10 Type = int
userinit8 Type = uint8
userinit16 Type = uint16
userinit64 Type = int64
userautoinit Type = int
intVar Type = int
```

Рис. 1.3 – Результат завдання 3.1

```
intVar = userinit16: Value = 2 Type = int
intVar = userautoinit: Value = -4 Type = int

Process finished with the exit code 0
```

Рис. 1.4 – Результат завдання 3.2

Завдання 4.

1. Виконати вираз: fmt.Println("c-- = ", c--)

Виконання завдання:

```
func main() {
// Ініціалізація змінних
var a int16 = 2
```

```
    var b int16 = 3
    var c int64 = 10

    fmt.Println("a + b        = ", a+b)
    fmt.Println("a - b        = ", a-b)
    fmt.Println("a * b        = ", a*b)
    fmt.Println("int(a / b)   = ", int(a/b), "\n")

    c--
    fmt.Println("c--     = ", c)
    c++
    fmt.Println("c++     = ", c)
    c += 10
    fmt.Println("c += 10 = ", c)
    c -= 5
    fmt.Println("c -= 5  = ", c)
    c *= 3
    fmt.Println("c *= 3  = ", c)
    c /= 7
    fmt.Println("c /= 7  = ", c)

    // Завдання.
    // 1. Виконати вираз: fmt.Println("c--     = ", c--)
    // Примітка: В Go не можна використовувати інкремент/декремент в виразах,
тільки як окремі оператори
    // Тому спочатку виконуємо декремент, потім виводимо
    c--
    fmt.Println("c--     = ", c)
}
```

Результат виконання:

```
c--     =  5
```

Рис. 1.5

Завдання 5.

1. Визначити розрядність платформи

Виконання завдання:

```
func main() {
  fmt.Println("Синоніми цілих типів\n")

  fmt.Println("byte    - int8")
  fmt.Println("rune    - int32")
  fmt.Println("int     - int32 або int64, в залежності від платформи")
  fmt.Println("uint    - uint32 або uint64, в залежності від платформи")

  // Завдання.
  // 1. Визначити розрядність платформи
  var testInt int
  var testUint uint

  fmt.Printf("\nРозрядність платформи:\n")
  fmt.Printf("int розмір: %d байт (%d біт)\n", unsafe.Sizeof(testInt),
unsafe.Sizeof(testInt)*8)
  fmt.Printf("uint розмір: %d байт (%d біт)\n", unsafe.Sizeof(testUint),
unsafe.Sizeof(testUint)*8)
  fmt.Printf("Архітектура: %s\n", runtime.GOARCH)

  if unsafe.Sizeof(testInt) == 8 {
    fmt.Println("Платформа: 64-бітна")
  } else {
    fmt.Println("Платформа: 32-бітна")
```

```
    }
}
```

Результат виконання:

```
byte    - int8
rune    - int32
int     - int32 або int64, в залежності від платформи
uint    - uint32 або uint64, в залежності від платформи

Розрядність платформи:
int розмір: 8 байт (64 біт)
uint розмір: 8 байт (64 біт)
Архітектура: amd64
Платформа: 64-бітна

Process finished with the exit code 0
```

Рис. 1.6

Завдання 6.

1. Пояснити результати операцій

Виконання завдання:

```
func main() {
    var x, y, z uint8

    x = 9
    y = 28
    z = x

    fmt.Println("Бітові операції")

    fmt.Printf("^x      = ^(%d)      = ^(%.8b)           = %.8b = %d\n", x, x, ^x, ^x)
    fmt.Printf("x << 2  = (%d << 2)  = (%.8b << 2)       = %.8b = %d\n", x, x, x<<2, x<<2)
    fmt.Printf("x >> 2  = (%d >> 2)  = (%.8b >> 2)       = %.8b = %d\n", x, x, x>>2, x>>2)
    fmt.Printf("x & y   = (%d & %d)  = (%.8b & %.8b)  = %.8b = %d\n", x, y, x, y, x&y, x&y)
```

```
fmt.Printf("x | y   = (%d | %d)  = (%.8b | %.8b)  = %.8b = %d\n", x, y, x, y, x|y, x|y)
fmt.Printf("x ^ y   = (%d ^ %d)  = (%.8b ^ %.8b)  = %.8b = %d\n", x, y, x, y, x^y, x^y)
fmt.Printf("x &^ y  = (%d &^ %d) = (%.8b &^ %.8b) = %.8b = %d\n", x, y, x, y, x&^y, x&^y)
fmt.Printf("x %% y   = (%d %% %d)  = (%.8b %% %.8b)  = %.8b = %d\n", x, y, x, y, x%y, x%y)

fmt.Println("\nБітові операції з присвоюванням")

x = z
x &= y
fmt.Printf("x &= y   = (%d &= %d)  = (%.8b &= %.8b)  = %.8b = %d\n", z, y, z, y, x, x)
x = z
x |= y
fmt.Printf("x |= y   = (%d |= %d)  = (%.8b |= %.8b)  = %.8b = %d\n", z, y, z, y, x, x)
x = z
x ^= y
fmt.Printf("x ^= y   = (%d ^= %d)  = (%.8b ^= %.8b)  = %.8b = %d\n", z, y, z, y, x, x)
x = z
x &^= y
fmt.Printf("x &^= y  = (%d &^= %d) = (%.8b &^= %.8b) = %.8b = %d\n", z, y, z, y, x, x)
x = z
x %= y
fmt.Printf("x %%= y   = (%d %%= %d)  = (%.8b %%= %.8b)  = %.8b = %d\n", z, y, z, y, x, x)
```

```
    // Завдання.
    // 1. Пояснити результати операцій
    fmt.Println("\nПояснення операцій:")
    fmt.Println("^x      - бітове НЕ (інверсія всіх бітів)")
    fmt.Println("x << 2  - зсув вліво на 2 біти (множення на 2^2 = 4)")
    fmt.Println("x >> 2  - зсув вправо на 2 біти (ділення на 2^2 = 4)")
    fmt.Println("x & y   - бітове І (AND) - 1 тільки якщо обидва біти = 1")
    fmt.Println("x | y   - бітове АБО (OR) - 1 якщо хоча б один біт = 1")
    fmt.Println("x ^ y   - бітове виключне АБО (XOR) - 1 якщо біти різні")
    fmt.Println("x &^ y  - бітове очищення (AND NOT) - очищає біти x де y=1")
    fmt.Println("x % y   - залишок від ділення")
}
```

Результат виконання:

```
Бітові операції
^x      = ^(9)      = ^(00001001)            = 11110110 = 246
x << 2  = (9 << 2)  = (00001001 << 2)        = 00100100 = 36
x >> 2  = (9 >> 2)  = (00001001 >> 2)        = 00000010 = 2
x & y   = (9 & 28)  = (00001001 & 00011100)  = 00001000 = 8
x | y   = (9 | 28)  = (00001001 | 00011100)  = 00011101 = 29
x ^ y   = (9 ^ 28)  = (00001001 ^ 00011100)  = 00010101 = 21
x &^ y  = (9 &^ 28) = (00001001 &^ 00011100) = 00000001 = 1
x % y   = (9 % 28)  = (00001001 % 00011100)  = 00001001 = 9

Бітові операції з присвоюванням
x &= y   = (9 &= 28)  = (00001001 &= 00011100)  = 00001000 = 8
x |= y   = (9 |= 28)  = (00001001 |= 00011100)  = 00011101 = 29
x ^= y   = (9 ^= 28)  = (00001001 ^= 00011100)  = 00010101 = 21
x &^= y  = (9 &^= 28) = (00001001 &^= 00011100) = 00000001 = 1
x %= y   = (9 %= 28)  = (00001001 %= 00011100)  = 00001001 = 9

Пояснення операцій:
^x      - бітове НЕ (інверсія всіх бітів)
x << 2  - зсув вліво на 2 біти (множення на 2^2 = 4)
x >> 2  - зсув вправо на 2 біти (ділення на 2^2 = 4)
x & y   - бітове І (AND) - 1 тільки якщо обидва біти = 1
x | y   - бітове АБО (OR) - 1 якщо хоча б один біт = 1
x ^ y   - бітове виключне АБО (XOR) - 1 якщо біти різні
x &^ y  - бітове очищення (AND NOT) - очищає біти x де y=1
x % y   - залишок від ділення

Process finished with the exit code 0
```

Рис. 1.7

Завдання 7.

1. Створіть 2 змінні різних типів. Виконайте арифметичні операції. Результат вивести в консоль

Виконання завдання:

```
func main() {
variable8 := int8(127)
variable16 := int16(16383)

fmt.Println("Приведення типів\n")

fmt.Printf("variable8        = %-5d = %.16b\n", variable8, variable8)
fmt.Printf("variable16       = %-5d = %.16b\n", variable16, variable16)
fmt.Printf("uint16(variable8) = %-5d = %.16b\n", uint16(variable8),
uint16(variable8))
fmt.Printf("uint8(variable16) = %-5d = %.16b\n", uint8(variable16),
uint8(variable16))

// Завдання.
// 1. Створіть 2 змінні різних типів. Виконайте арифметичні операції. Результат
вивести в консоль
var a int32 = 100
var b int64 = 50

fmt.Println("\nАрифметичні операції з різними типами:")
fmt.Printf("a (int32) = %d\n", a)
fmt.Printf("b (int64) = %d\n", b)

// Приведення типів для виконання операцій
fmt.Printf("int64(a) + b = %d\n", int64(a)+b)
fmt.Printf("int64(a) - b = %d\n", int64(a)-b)
```

```
    fmt.Printf("int64(a) * b = %d\n", int64(a)*b)
    fmt.Printf("int64(a) / b = %d\n", int64(a)/b)
    fmt.Printf("int64(a) %% b = %d\n", int64(a)%b)
}
```

Результат виконання:

```
Приведення типів

variable8         = 127   = 0000000001111111
variable16        = 16383 = 0011111111111111
uint16(variable8) = 127   = 0000000001111111
uint8(variable16) = 255   = 0000000011111111

Арифметичні операції з різними типами:
a (int32) = 100
b (int64) = 50
int64(a) + b = 150
int64(a) - b = 50
int64(a) * b = 5000
int64(a) / b = 2
int64(a) % b = 0

Process finished with the exit code 0
```

Рис. 1.8

Завдання 8.

1. Створіть змінні різних типів, використовуючи короткий запис та ініціалізацію за замовчуванням. Результат вивести в консоль

Виконання завдання:

```
func main() {
var defaultFloat float32
var defaultDouble float64 = 5.5

fmt.Println("defaultfloat      = ", defaultFloat)
fmt.Printf("defaultDouble (%T) = %f\n\n", defaultDouble, defaultDouble)

fmt.Println("MAX float32       = ", math.MaxFloat32)
fmt.Println("MIN float32       = ", math.SmallestNonzeroFloat32, "\n")
```

```
fmt.Println("MAX float64       = ", math.MaxFloat64)
fmt.Println("MIN float64       = ", math.SmallestNonzeroFloat64, "\n")

// Завдання.
// 1. Створіть змінні різних типів, використовуючи короткий запис та ініціалізацію за замовчуванням. Результат вивести в консоль
fmt.Println("Змінні з коротким записом:")
shortInt := 42
shortFloat := 3.14
shortString := "Привіт"
shortBool := true

fmt.Printf("shortInt (%T) = %d\n", shortInt, shortInt)
fmt.Printf("shortFloat (%T) = %f\n", shortFloat, shortFloat)
fmt.Printf("shortString (%T) = %s\n", shortString, shortString)
fmt.Printf("shortBool (%T) = %t\n\n", shortBool, shortBool)

fmt.Println("Змінні з ініціалізацією за замовчуванням:")
var defaultInt int
var defaultFloat32 float32
var defaultFloat64 float64
var defaultString string
var defaultBool bool

fmt.Printf("defaultInt (%T) = %d\n", defaultInt, defaultInt)
fmt.Printf("defaultFloat32 (%T) = %f\n", defaultFloat32, defaultFloat32)
fmt.Printf("defaultFloat64 (%T) = %f\n", defaultFloat64, defaultFloat64)
fmt.Printf("defaultString (%T) = '%s'\n", defaultString, defaultString)
```

```
    fmt.Printf("defaultBool (%T) = %t\n", defaultBool, defaultBool)
}
```

Результат виконання:

```
defaultfloat       =  0
defaultDouble (float64) = 5.500000

MAX float32        =  3.4028234663852886e+38
MIN float32        =  1.401298464324817e-45

MAX float64        =  1.7976931348623157e+308
MIN float64        =  5e-324

Змінні з коротким записом:
shortInt (int) = 42
shortFloat (float64) = 3.140000
shortString (string) = Привіт
shortBool (bool) = true

Змінні з ініціалізацією за замовчуванням:
defaultInt (int) = 0
defaultFloat32 (float32) = 0.000000
defaultFloat64 (float64) = 0.000000
defaultString (string) = ''
defaultBool (bool) = false

Process finished with the exit code 0
```

Рис. 1.9

Завдання 9.

1. Пояснити результати операцій

Виконання завдання:

```
func main() {
    var first, second bool
    var third bool = true
    fourth := !third
    var fifth = true

    fmt.Println("first  = ", first)       // false
    fmt.Println("second = ", second)      // false
    fmt.Println("third  = ", third)       // true
    fmt.Println("fourth = ", fourth)      // false
```

```
fmt.Println("fifth  = ", fifth, "\n") // true

fmt.Println("!true  = ", !true)        // false
fmt.Println("!false = ", !false, "\n") // true

fmt.Println("true && true   = ", true && true)         // true
fmt.Println("true && false  = ", true && false)        // false
fmt.Println("false && false = ", false && false, "\n") // false

fmt.Println("true || true   = ", true || true)         // true
fmt.Println("true || false  = ", true || false)        // true
fmt.Println("false || false = ", false || false, "\n") // false

fmt.Println("2 < 3  = ", 2 < 3)        // true
fmt.Println("2 > 3  = ", 2 > 3)        // false
fmt.Println("3 < 3  = ", 3 < 3)        // false
fmt.Println("3 <= 3 = ", 3 <= 3)       // true
fmt.Println("3 > 3  = ", 3 > 3)        // false
fmt.Println("3 >= 3 = ", 3 >= 3)       // true
fmt.Println("2 == 3 = ", 2 == 3)       // false
fmt.Println("3 == 3 = ", 3 == 3)       // true
fmt.Println("2 != 3 = ", 2 != 3)       // true
fmt.Println("3 != 3 = ", 3 != 3, "\n") // false

// Завдання.
// 1. Пояснити результати операцій
fmt.Println("Пояснення результатів операцій:")
fmt.Println("\nЛогічні операції:")
fmt.Println("! - логічне НЕ (інверсія): !true = false, !false = true")
fmt.Println("&& - логічне І (AND): повертає true тільки якщо обидва операнди
```

```
true")
    fmt.Println("|| - логічне АБО (OR): повертає true якщо хоча б один операнд true")
    fmt.Println("\nОперації порівняння:")
    fmt.Println("<  - менше")
    fmt.Println(">  - більше")
    fmt.Println("<= - менше або дорівнює")
    fmt.Println(">= - більше або дорівнює")
    fmt.Println("== - дорівнює")
    fmt.Println("!= - не дорівнює")
}
```

Результат виконання:

```
true || true   =  true
true || false  =  true
false || false =  false

2 < 3  =  true
2 > 3  =  false
3 < 3  =  false
3 <= 3 =  true
3 > 3  =  false
3 >= 3 =  true
2 == 3 =  false
3 == 3 =  true
2 != 3 =  true
3 != 3 =  false

Пояснення результатів операцій:

Логічні операції:
! - логічне НЕ (інверсія): !true = false, !false = true
&& - логічне І (AND): повертає true тільки якщо обидва операнди true
|| - логічне АБО (OR): повертає true якщо хоча б один операнд true

Операції порівняння:
<  - менше
>  - більше
<= - менше або дорівнює
>= - більше або дорівнює
== - дорівнює
!= - не дорівнює

Process finished with the exit code 0
```

Рис. 1.10

Завдання 10.

1. Вивести українську літеру 'Ї'
2. Пояснити призначення типу "rune"

Виконання завдання:

```
func main() {
var chartype int8 = 'R'

fmt.Printf("Code '%c' - %d\n", chartype, chartype)

// Завдання.
// 1. Вивести українську літеру 'Ї'
var ukrainianLetter rune = 'Ї'
fmt.Printf("Code '%c' - %d\n", ukrainianLetter, ukrainianLetter)

// Альтернативний варіант з int8 (але 'Ї' не поміститься в int8, тому використо-
вуємо rune)
var ukrainianLetterRune rune = 'Ї'
fmt.Printf("Українська літера '%c' має код: %d\n", ukrainianLetterRune, ukrainian-
LetterRune)

// 2. Пояснити призначення типу "rune"
fmt.Println("\nПояснення типу 'rune':")
fmt.Println("rune - це псевдонім для int32 в Go")
fmt.Println("Призначений для зберігання Unicode код-пойнтів (символів)")
fmt.Println("Дозволяє коректно працювати з багатобайтовими символами
(наприклад, українськими літерами)")
fmt.Println("На відміну від int8 (byte), який може зберігати тільки 1 байт (0-
255),")
fmt.Println("rune може зберігати будь-який Unicode символ (до 4 байт)")
}
```

Результат виконання:

```
Code 'R' - 82
Code 'Ї' - 1031
Українська літера 'Ї' має код: 1031
```

Рис. 1.11 – Результат завдання 10.1

```
Пояснення типу 'rune':
rune - це псевдонім для int32 в Go
Призначений для зберігання Unicode код-пойнтів (символів)
Дозволяє коректно працювати з багатобайтовими символами (наприклад, українськими літерами)
На відміну від int8 (byte), який може зберігати тільки 1 байт (0-255),
rune може зберігати будь-який Unicode символ (до 4 байт)

Process finished with the exit code 0
```

Рис. 1.12 – Результат завдання 10.2

Висновок

Під час виконання роботи було вивчено принципи застосування бітових операцій у мові Go. Встановлено, що ці операції здійснюються безпосередньо над кожним бітом операнда окремо. Також було практично опрацьовано використання побітових операторів у поєднанні з операторами присвоювання для оптимізації коду.

Контрольні питання

1. Які основні типи даних існують у Go та їхні властивості?

У мові Go виділяють наступні базові категорії типів:

- **Цілочисельні (int, int64, uint тощо):** Використовуються для збереження цілих чисел. Бувають знаковими та беззнаковими.
- **Числа з рухомою комою (float32, float64):** Призначені для роботи з дробовими числами.
- **Логічний тип (bool):** Приймає лише два значення: true або false.
- **Рядкові (string):** Послідовності символів у кодуванні UTF-8.
- **Комплексні числа (complex64, 128):** Для математичних обчислень з уявною частиною.

Ключові властивості:

1. **Сувора типізація:** Go не дозволяє змішувати різні типи (наприклад, int та float64) в операціях без явного приведення.

2. **Статичність:** Тип змінної визначається під час компіляції та не змінюється в процесі виконання.
3. **Нульові значення:** Якщо змінна не ініціалізована, вона автоматично отримує «дефолтне» значення (0 для чисел, "" для рядків).

---

2. Опишіть базову структуру програми на Go

Типова програма складається з трьох основних частин:

Go

```
package main  // 1. Оголошення пакету

import "fmt"  // 2. Імпорт необхідних модулів

func main() { // 3. Головна функція (точка входу)
   fmt.Println("Hello, Go!")
}
```

- **package main:** Вказує компілятору, що цей файл є виконуваним файлом, а не бібліотекою.
- **import:** Дозволяє використовувати функціонал з інших пакетів.
- **func main():** Спеціальна функція, з якої починається виконання коду.

---

3. Класифікація цілочисельних типів даних

Цілочисельні типи поділяються на дві групи залежно від наявності знаку:

Категорія	**Типи**	**Особливість**
Знакові	int8, int16, int32, int64, int	Можуть бути як додатними, так і від'ємними.
Беззнакові	uint8, uint16, uint32, uint64, uint	Тільки додатні значення та нуль.
Спеціальні	byte (аліас uint8), rune (аліас int32)	Використовуються для символів та байтових масивів.

4. У чому полягає різниця між різними цілочисельними типами?

Основні відмінності базуються на трьох критеріях:

1. **Розрядність (розмір у пам'яті):** Визначає, скільки біт займає число (наприклад, int8 — 8 біт, int64 — 64 біти).
2. **Діапазон значень:** Напряму залежить від розрядності. Наприклад, int8 вміщує значення від -128 до 127, тоді як uint8 — від 0 до 255.
3. **Архітектурна залежність:** Типи int та uint мають розмір, що відповідає розрядності процесора (32 або 64 біти).

---

5. Як реалізовано роботу з рядками (strings) у Go?

Рядки в Go мають декілька специфічних особливостей:

- **Внутрішнє представлення:** Рядок — це незмінна послідовність байтів, зазвичай закодованих у **UTF-8**.
- **Незмінність (Immutability):** Після створення вміст рядка не можна змінити. Не можна написати str[0] = 'A', потрібно створювати новий рядок.
- **Синтаксис:** Використовуються подвійні лапки " " для звичайних рядків або зворотні лапки ` ` для багаторядкового тексту (raw strings).
- **Гнучкість:** Завдяки UTF-8, Go "з коробки" підтримує будь-які мови та символи (емодзі, ієрогліфи тощо).